

文献阅读笔记

一. 文章摘要：

- 提出现有系统在执行 LM Programs 时有编程困难和执行低效的问题，故论文提出 SGLang 作为用于高效编程和执行结构化 LM 程序的系统。其主要技术贡献包括：RadixAttention, Compressed FSM 与 API 推测执行，并在文中解析其机制。实验在多种模型与硬件上进行评估，得出 SGLang 相比现有的编程和推理系统，实现了大量的吞吐量提升，延迟降低，缓存命中率提高，多模态任务效率提高的结论

二. 技术解析：

1. RadixAttention：

- KV 缓存复用，解决现有推理引擎在请求完成后丢弃 KV 缓存，导致多个共享前缀的调用重复计算的问题。
- 用基数树管理 KV 缓存，边可标记多 token 序列；采用 LRU 驱逐（先驱逐叶子，保留公共祖先）；节点带引用计数，运行中的请求不可驱逐；缓存感知调度按匹配前缀长度排序，近似 DFS 顺序，论文证明该顺序在缓存足够时可达最优命中率。

2. 压缩状态有限机 Compressed FSM：

- 快速约束解码，解决某些位置只有一个合法 token 也要多次前向传递
- 将正则表达式转为字符级 FSM，把相邻单一转移边合并为压缩边；解码时若处于压缩边，可一次前向传递解出多个 token (Jump Forward)；用原始 tokenizer 重新分词以避免分词伪影。

3. API 推测执行：

- 针对黑盒 API 模型优化，解决仅 API 可访问的模型，多调用程序重复支付输入 token 成本。
- 第一次调用时忽略停止条件，让模型多生成一些 token；解释器保留这些输出，在后续原语中匹配复用；配合少样本提示提高匹配率。

三. 实验结果：

通过多维度评估，生产部署验证，开销分析，消融研究等得出以下结论：

- 吞吐量最高提升 6.4 倍，延迟最高降低 3.7 倍
- 缓存命中率 50%–99%，调度达最优命中率的 96%。
- RadixAttention 无命中时开销 < 0.3%。
- 压缩 FSM 使 JSON 解码吞吐量提升 1.6 倍。
- 已部署于 Chatbot Arena, Vicuna-33B 首 token 延迟降低 1.7 倍。

四. 启发总结：

- KV 缓存是 LLM 推理系统中重要的资源，操作系统中的 LRU、引用计数、前缀树等经典思想可以直接迁移过来；同时，前端语言层与后端运行时协同设计，能提供运行时无法自行推断的信息，从而简化调度与缓存匹配；此外，当输出约束足够强时，解码不必逐 token 进行，可以一次生成多个 token；即使是黑盒 API 模型，也能通过推测执行和提示工程降低调用成本。